perplexity

great - and we can add in a cotrastive loss across a batch to help encodings vary and fill a full expressability range for training faster. (different observation encoders of the same head should land at the same representation, while not matching inputs from different time stamps)

Exactly — that's a perfect use of **contrastive self-supervised learning** on top of the multi-view latent SSM. You're describing two complementary constraints:

1. **Cross-view agreement at the same time:**
   Encodings from different observation heads at the same timestamp should collapse to the same latent representation.
   → "Different encoders of the same head should land at the same representation."
2. **Temporal discriminability:**
   Encodings from different timestamps should be distinguishable in latent space.
   → "Not matching inputs from different time stamps."

That's essentially a **multi-view, temporal contrastive loss** on the latent, very much in the spirit of contrastive predictive coding and multi-view SSL. [1] [2] [3] [4] [5]

Below is how to plug that in cleanly.

1. What you want the contrastive loss to enforce

Given a batch of timesteps $t = 1, \dots, B$:

- For each timestep t, you have multiple views:
  - $\mathbf{y}_t^{(\text{SD})}$
  - $\mathbf{y}_t^{(\text{HF})}$
  - optionally $\mathbf{y}_t^{(\text{idx})}$
- Each view has an encoder:

$$mathbfz^{(i)}\_t = e^{(i)}(mathbfy^{(i)}\_t), quadiintextSD, textHF, textidx$$

You want:

(A) Same-time, cross-view agreement

For a fixed t, all $\mathbf{z}_t^{(i)}$ should be close:

$$\mathcal{L}_{\text{agree}} = \sum_t \sum_{i<j} \left\| \mathbf{z}_t^{(i)} - \mathbf{z}_t^{(j)} \right\|^2$$

You can either:

- Add this as an explicit MSE-style term, or
- Enforce it implicitly via contrastive loss by treating different views at the same time as **positives**.

(B) Different-time discrimination

For $t \neq t'$, the shared latent representations should be distinguishable:

$$\mathbf{z}_t \not\approx \mathbf{z}_{t'}$$

This is where **contrastive loss** across the batch helps: treat each timestep's shared latent as a unique "identity" and push different timesteps apart.

Combining these gives you:

- Strong pressure to use the latent capacity (fill the representation space).
- Faster training because the model gets a clearer self-supervised signal about what should and shouldn't match.

2. Define a shared latent per timestep

To keep things simple, define a single shared latent per timestep as before, e.g.:

$$\mathbf{z}_t = \frac{1}{V} \sum_i \mathbf{z}_t^{(i)}$$

or use a learned fusion (attention, gating). The contrastive loss will operate on these $\mathbf{z}_t$.

You still keep:

- **Reconstruction loss** from $\mathbf{z}_t$ to each view.
- **Dynamics loss** for forecastability.
- Now plus **contrastive loss** over $\{\mathbf{z}_t\}_{t=1}^B$.

3. Contrastive loss design

3.1 Positives and negatives

For a batch of timesteps $t = 1, \dots, B$:

- **Anchor:** $\mathbf{z}_t$
- **Positives:**
  - Other views at the same time: $\mathbf{z}_t^{(i)}$ for all i.
  - Optionally, temporally nearby timesteps if you want smoothness (e.g., $t \pm 1$) — but you said you *don't* want different timestamps to match, so we'll keep positives strictly same-time, cross-view.
- **Negatives:**
  - All $\mathbf{z}_{t'}$ for $t' \neq t$.
  - Optionally, other views at other times: $\mathbf{z}_{t'}^{(i)}$.

This matches your requirement: "different observation encoders of the same head should land at the same representation, while not matching inputs from different time stamps."

3.2 InfoNCE-style loss on latents

A standard choice is **InfoNCE** (used in contrastive predictive coding and many SSL methods): [2] [3] [1]

For each anchor t:

$$\mathcal{L}_{\text{cont}}^{(t)} = -\sum_{p \in \mathcal{P}(t)} \log \frac{\exp(\text{sim}(\mathbf{z}_t, \mathbf{z}_p)/\tau)}{\sum_{n \in \mathcal{N}(t)} \exp(\text{sim}(\mathbf{z}_t, \mathbf{z}_n)/\tau)}$$

where:

- $\mathcal{P}(t)$ = positives for t (other views at same time).
- $\mathcal{N}(t)$ = all candidates in the batch (including positives).
- $\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$ (cosine similarity).
- τ = temperature (e.g., 0.07–0.2).

Then:

$$\mathcal{L}_{\text{cont}} = \frac{1}{B} \sum_t \mathcal{L}_{\text{cont}}^{(t)}$$

This:

- Pulls same-time, cross-view latents together.
- Pushes different-time latents apart.
- Encourages the model to use the latent dimensions effectively (good for "full expressability range").

If you want to be explicit about views, you can define positives as all pairs (i, j) at the same time and negatives as all pairs with $t \neq t'$, but the simpler version (one shared $\mathbf{z}_t$ per time, contrast across time) is usually enough.

4. Full objective with contrastive term

Your total loss becomes:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda_{\text{dyn}}\mathcal{L}_{\text{dyn}} + \lambda_{\text{cont}}\mathcal{L}_{\text{cont}} + \lambda_{\text{reg}}\mathcal{L}_{\text{reg}}$$

Where:

- $\mathcal{L}_{\text{recon}}$: multi-view reconstruction (as before).
- $\mathcal{L}_{\text{dyn}}$: latent dynamics predictability.
- $\mathcal{L}_{\text{cont}}$: contrastive loss across batch (same-time agreement, different-time separation).
- $\mathcal{L}_{\text{reg}}$: optional regularisation (latent smoothness, L2, etc.).

You can also keep an explicit **cross-view agreement** term if you like:

$$\mathcal{L}_{\text{agree}} = \sum_t \sum_{i<j} \|\mathbf{z}_t^{(i)} - \mathbf{z}_t^{(j)}\|^2$$

and then:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda_{\text{agree}}\mathcal{L}_{\text{agree}} + \lambda_{\text{dyn}}\mathcal{L}_{\text{dyn}} + \lambda_{\text{cont}}\mathcal{L}_{\text{cont}} + \dots$$

Often, $\mathcal{L}_{\text{cont}}$ alone is enough to enforce cross-view agreement, so you can start without $\mathcal{L}_{\text{agree}}$ and add it only if needed.

5. PyTorch-style implementation sketch

Assume:

- `z_shared[t]` = shared latent for timestep `t` (from averaging encoder outputs).
- `z_view[t][i]` = per-view latent for view `i` at time `t`.

5.1 Contrastive loss function

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def contrastive_loss_latents(
    z_shared,        # Tensor [B, D]
    z_views,         # List[Tensor] of length V, each [B, D]
    tau=0.1
):
    """
    InfoNCE-style contrastive loss:
      - Positives: different views at same time
      - Negatives: different times (same shared latent)
```

```
    z_shared: [B, D]
    z_views: list of V tensors, each [B, D]
    """
    B, D = z_shared.shape
    device = z_shared.device

    # Normalize shared latents for cosine similarity
    z_shared_norm = F.normalize(z_shared, p=2, dim=1)  # [B, D]

    # Similarity matrix between shared latents: [B, B]
    sim_matrix = torch.matmul(z_shared_norm, z_shared_norm.t()) / tau

    # For each anchor t, positives are other views at same t.
    # We'll construct a positive mask and use InfoNCE.

    # Compute average similarity to positives (other views at same time)
    # For simplicity, treat each view's latent at time t as a positive key.
    # We'll compute sim(z_shared[t], z_views[i][t]) for all i.

    # Stack views: [V, B, D]
    z_views_stack = torch.stack(z_views, dim=0)  # [V, B, D]
    z_views_norm = F.normalize(z_views_stack, p=2, dim=2)  # [V, B, D]

    # Compute similarity between z_shared[t] and each view at same t:
    # result: [V, B]
    sim_pos = torch.einsum('bd,vbd->vb', z_shared_norm, z_views_norm) / tau

    # Exclude self-similarity if one of the views is effectively the same as z_shared
    # If z_shared is the mean of views, it's not identical to any single view, so we

    # For InfoNCE, for each anchor t:
    #   numerator = sum_{p in positives} exp(sim(t, p))
    #   denominator = sum_{n in all candidates} exp(sim(t, n))
    # Here, candidates = all times' shared latents.

    # Numerator: sum over views of exp(sim_pos[v, t])
    exp_sim_pos = torch.exp(sim_pos)                # [V, B]
    numerator = exp_sim_pos.sum(dim=0)              # [B]

    # Denominator: sum over all times n of exp(sim_matrix[t, n])
    exp_sim_all = torch.exp(sim_matrix)             # [B, B]
    denominator = exp_sim_all.sum(dim=1)            # [B]

    # InfoNCE loss per anchor
    loss_per_anchor = -torch.log(numerator / (denominator + 1e-8) + 1e-8)
    loss = loss_per_anchor.mean()

    return loss
```

This implements:

- **Positives:** all views at the same time.
- **Negatives:** all other times' shared latents.

If you want stricter control (e.g., exclude a particular view from positives, or include temporally adjacent negatives), you can adjust the numerator/denominator masks accordingly.

5.2 Integrating into the training step

Extend your `train_step_selfsup` to compute per-view latents, shared latent, and then the contrastive loss.

```
def train_step_selfsup_contrastive(
    model,
    y_sd_list,
    y_hf_link_list,
    y_hf_height_list,
    u_list,
    y_idx_list=None,
    optimizer=None,
    device="cpu",
    w_sd=1.0,
    w_hf_link=0.5,
    w_hf_height=0.5,
    w_idx=0.0,
    lambda_dyn=1.0,
    lambda_cont=0.5,
    lambda_smooth=0.1,
    tau=0.1,
):
    model.train()
    if optimizer is not None:
        optimizer.zero_grad()

    T = len(y_sd_list)
    B = y_sd_list[^0].shape[^0]

    # Storage for latents
    z_shared_list = []
    z_views_list = []  # list of per-timestep lists of view latents

    # First pass: encode all timesteps
    for t in range(T):
        y_sd = y_sd_list[t]
        y_hf_link = y_hf_link_list[t]
        y_hf_height = y_hf_height_list[t]
        y_idx = y_idx_list[t] if y_idx_list is not None else None

        # Per-view latents
        z_sd = model.encoders["sd"](y_sd)
        z_hf = model.encoders["hf"](
            torch.cat([y_hf_link.unsqueeze(-1), y_hf_height.unsqueeze(-1)], dim=-1)
        )
        views = [z_sd, z_hf]
        if y_idx is not None and "idx" in model.encoders:
            z_idx = model.encoders["idx"](y_idx)
            views.append(z_idx)

        # Shared latent as mean
```

```
        z_shared = torch.stack(views, dim=0).mean(dim=0)  # [B, D]

        z_shared_list.append(z_shared)
        z_views_list.append(views)

    # Stack for easier indexing: [T, B, D]
    z_shared_stack = torch.stack(z_shared_list, dim=0)
    # z_views_list is T x (V x [B, D])

    total_loss = 0.0
    total_recon = 0.0
    total_dyn = 0.0
    total_cont = 0.0
    total_smooth = 0.0

    # Reconstruction + dynamics + smoothness loop
    z_prev = None
    for t in range(T):
        z = z_shared_stack[t]  # [B, D]

        # Decode
        y_sd_pred = model.decode_sd(z)
        link_logit, height_pred = model.decode_hf(z)
        y_idx_pred = model.decode_idx(z) if (y_idx_list is not None and model.decoder

        y_sd = y_sd_list[t]
        y_hf_link = y_hf_link_list[t]
        y_hf_height = y_hf_height_list[t]
        y_idx = y_idx_list[t] if y_idx_list is not None else None

        # Reconstruction losses
        loss_sd = ((y_sd_pred - y_sd)**2).mean()
        loss_link = nn.functional.binary_cross_entropy_with_logits(
            link_logit, y_hf_link.float()
        )
        loss_height = ((height_pred - y_hf_height)**2).mean()

        loss_recon = w_sd * loss_sd + w_hf_link * loss_link + w_hf_height * loss_heig
        if y_idx is not None and y_idx_pred is not None:
            loss_idx = ((y_idx_pred - y_idx)**2).mean()
            loss_recon = loss_recon + w_idx * loss_idx

        # Dynamics loss
        if t < T - 1:
            with torch.no_grad():
                z_next = z_shared_stack[t+1]
            u = u_list[t]
            z_next_pred = model.step_latent(z, u)
            loss_dyn = ((z_next_pred - z_next)**2).mean()
        else:
            loss_dyn = torch.tensor(0.0, device=device)

        # Smoothness
        if z_prev is not None:
            loss_smooth = ((z - z_prev)**2).mean()
        else:
```

```
            loss_smooth = torch.tensor(0.0, device=device)

        loss = loss_recon + lambda_dyn * loss_dyn + lambda_smooth * loss_smooth

        if optimizer is not None:
            loss.backward()

        total_loss += loss.item()
        total_recon += loss_recon.item()
        total_dyn += loss_dyn.item()
        total_smooth += loss_smooth.item()

        z_prev = z

    # Contrastive loss over all timesteps
    # Build per-timestep view lists for the contrastive function
    z_views_by_time = []
    for t in range(T):
        views_t = z_views_list[t]  # list of V tensors [B, D]
        z_views_by_time.append(views_t)

    # We want to compute contrastive loss per timestep and average.
    # We'll loop over t and call contrastive_loss_latents for each.
    cont_losses = []
    for t in range(T):
        z_shared_t = z_shared_list[t]  # [B, D]
        views_t = z_views_list[t]      # list of [B, D]
        loss_cont_t = contrastive_loss_latents(z_shared_t, views_t, tau=tau)
        cont_losses.append(loss_cont_t)

    loss_cont = torch.stack(cont_losses).mean()

    # Add contrastive gradient
    if optimizer is not None:
        (lambda_cont * loss_cont).backward()
        optimizer.step()

    total_cont = loss_cont.item()
    total_loss += lambda_cont * total_cont

    return {
        "loss": total_loss / T,
        "recon": total_recon / T,
        "dyn": total_dyn / T,
        "cont": total_cont,
        "smooth": total_smooth / T,
    }
```

Key points:

- The contrastive term is computed **once per batch** over all timesteps, using the per-timestep shared latent and per-view latents.
- It explicitly:
    - Pulls different views at the same time together.

    - Pushes different times apart.
- You can tune `lambda_cont` and `tau` to control how strongly this shapes the latent.

6. How this helps training

- **Faster convergence:** The contrastive term gives a strong, global self-supervised signal about which representations should/shouldn't match, reducing ambiguity.[3] [1] [2]
- **Better use of capacity:** By forcing different timesteps to be separable, the model is encouraged to spread representations across the latent space ("fill a full expressability range").
- **Consistent with your physics goal:** You still have:
    - Reconstruction → good transport/forward models.
    - Dynamics → forecastable latent.
    - Contrastive → well-structured, discriminative latent.

If you share your intended latent dimension and batch size (e.g., how many timesteps you train over at once), I can suggest concrete values for `lambda_cont`, `tau`, and any normalisation tricks to keep training stable.

⁂

1. https://proceedings.mlr.press/v202/rodri-guez-galvez23a/rodri-guez-galvez23a.pdf
2. https://openreview.net/pdf?id=-bdp_8Itjwp
3. https://machinelearning.apple.com/research/entropy-reconstruction
4. https://openreview.net/forum?id=ONfWFluZBI
5. https://arxiv.org/html/2410.14673v1